

Agile Flow - Final Team Presentation

A Productivity Visualization Agile Methodologies Tracker

Team: Andrew MacRossie, Carolina Perez, Erick Samayoa , Mike Davis, Sergio Rojas

Course: CSPB 3308 Software Development Tools & Methods

Problem Motivation

- Students struggle to develop Agile Methodologies:

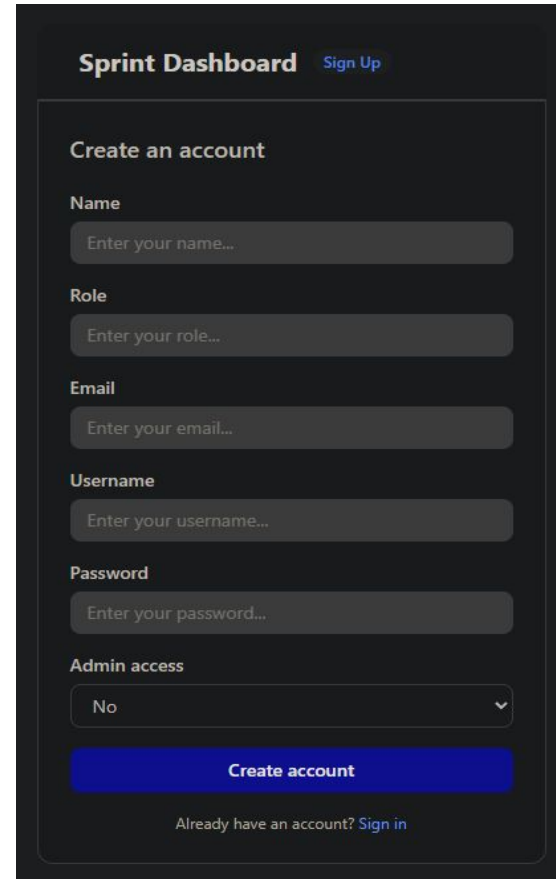
- Pay-walling from current Agile productivity applications
- Limited access to Agile implementation
- No shared task tracking/limited resources to do so
- Students struggle to effectively apply agile methodologies

- Goals

- Reduce friction in team collaboration
- Streamline Agile-based project coordination

What Agile Flow Does

- Agile Flow enables groups to:
 - Create and join projects
 - Maintain shared task boards
 - Creating projects, sprints and tasks
 - Burn down/up Chart visualize
 - scope creep
 - Track Sprint Progress
 - completion dates

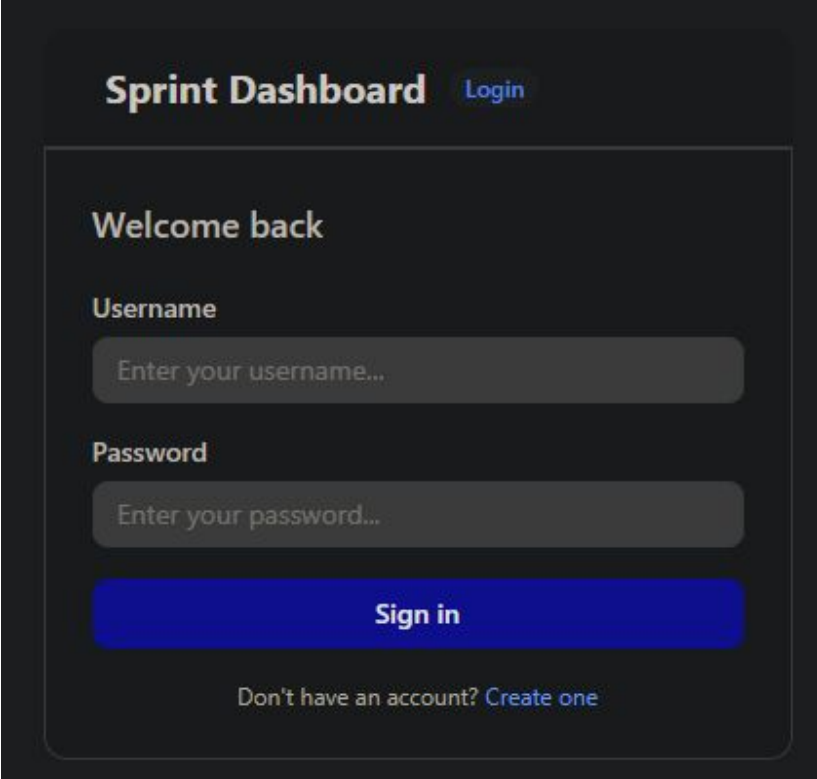
A screenshot of a web application's sign-up interface. At the top, it says 'Sprint Dashboard' with a 'Sign Up' link. Below this is a 'Create an account' section. It contains several input fields: 'Name' (placeholder: 'Enter your name...'), 'Role' (placeholder: 'Enter your role...'), 'Email' (placeholder: 'Enter your email...'), 'Username' (placeholder: 'Enter your username...'), and 'Password' (placeholder: 'Enter your password...'). There is also a dropdown menu for 'Admin access' currently set to 'No'. A blue 'Create account' button is at the bottom of the form. Below the button, it says 'Already have an account? Sign in'.

Main dashboard showing active groups and tasks.

Demo Flow Overview

- Our live demo will follow this sequence:

1. Display Login Page
2. Display Create Account Page
3. Fetch current user data (mock login)
4. Dashboard Overview
 - a. Create a Project
 - b. Create a Sprint
 - c. Create Multiple Tasks
 - d. Update Task Completeness
 - e. Delete Tasks
 - f. Delete a Sprint



The screenshot shows a dark-themed login interface. At the top, it says 'Sprint Dashboard' in white, with a 'Login' link in blue to its right. Below this is a 'Welcome back' message. The form contains two input fields: 'Username' with a placeholder 'Enter your username...' and 'Password' with a placeholder 'Enter your password...'. A blue 'Sign in' button is positioned below the password field. At the bottom, there is a link that says 'Don't have an account? Create one'.

Login screen with
authentication flow.

System Architecture Overview

- FRONTEND:

- React.JS w/Tailwind CSS
- ChartsJS

- BACKEND:

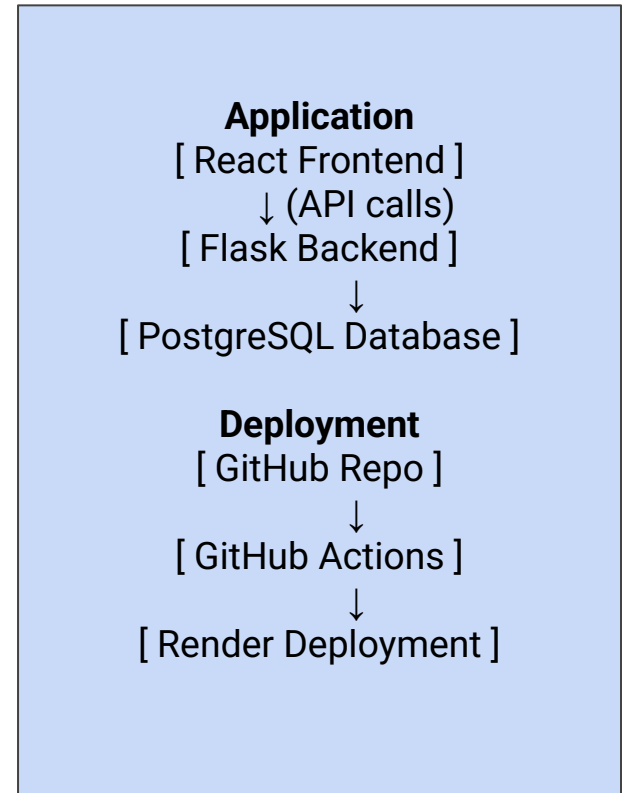
- Flask RESTful API

- DATABASE:

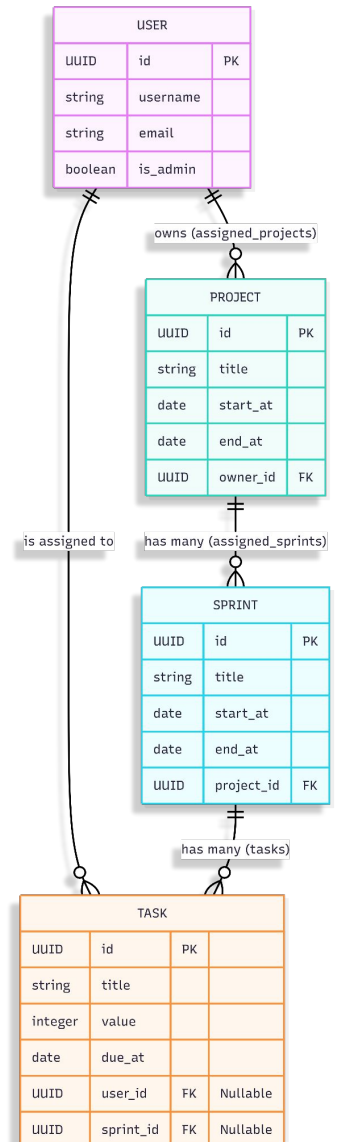
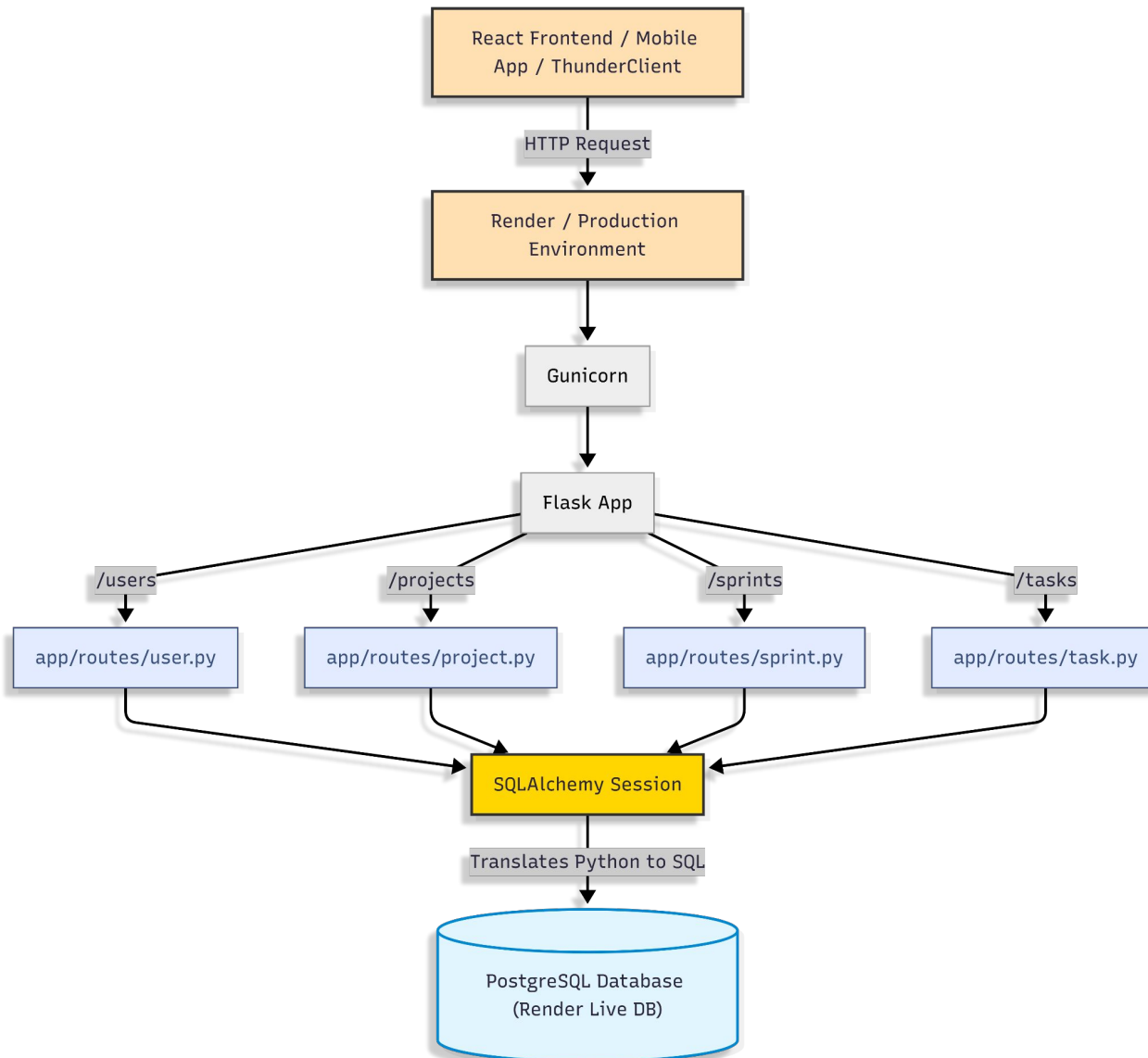
- PostgreSQL - SQLAlchemy
- ORM

- CI/CD:

- GitHub Actions → Render



Architecture Diagram



Key Features Implemented

Agile Flow Running

Sprint Dashboard Sprint 3

Projects Dashboard Page

Burndown Chart



Burndown Progress Chart

Project Creation

Task Create
Task Updating
Task deletion

Sprint Creation
Sprint Deletion

Projects

New project

Project name

Add Project

Agile Backend API

Initialize Project

Sprint: 14 days

At risk

0%

- ☐ Init DB Script
- ☐ Environment Setup
- ☐ MoSCoW Prioritization
- ☐ PLANNING POKER

New task...

+ Add

User Stories & DB

Sprint: 12 days

At risk

0%

- ☐ Add Email Notifications
- ☐ Write Documentation
- ☐ Project GET Route
- ☐ User POST Route

New task...

+ Add

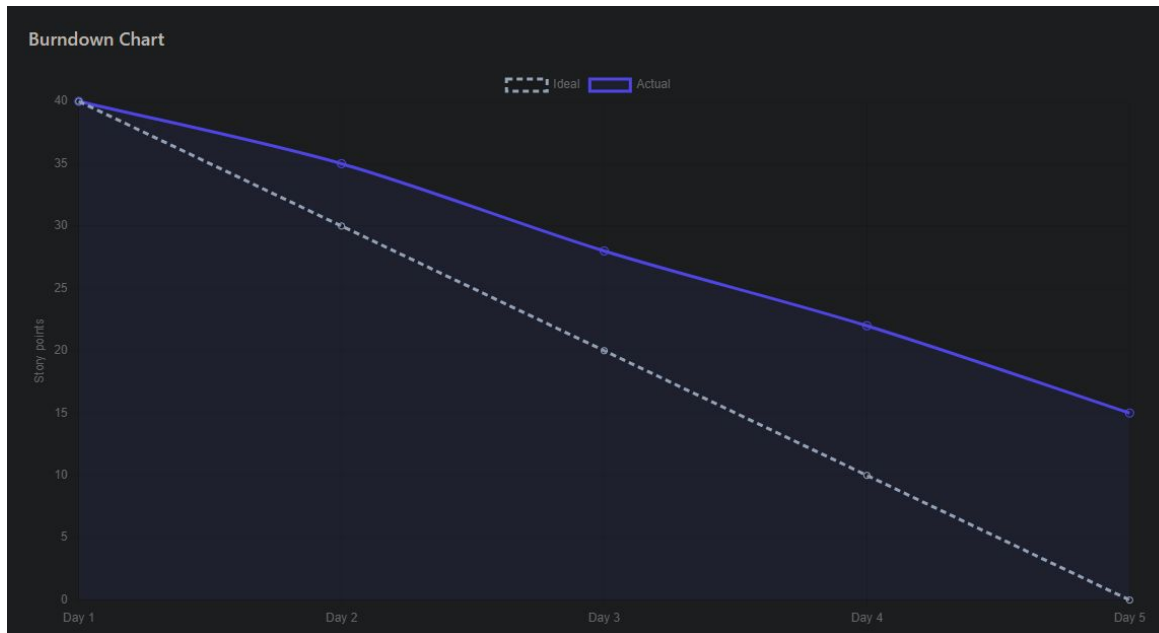
New sprint...

+ Add

UI Responsive fetches

Burndown Chart Feature

- Visualizes sprint progress over time
- Compares planned vs completed work
- Helps team track velocity and workload (tasks/sprints)
- Updates when tasks are completed

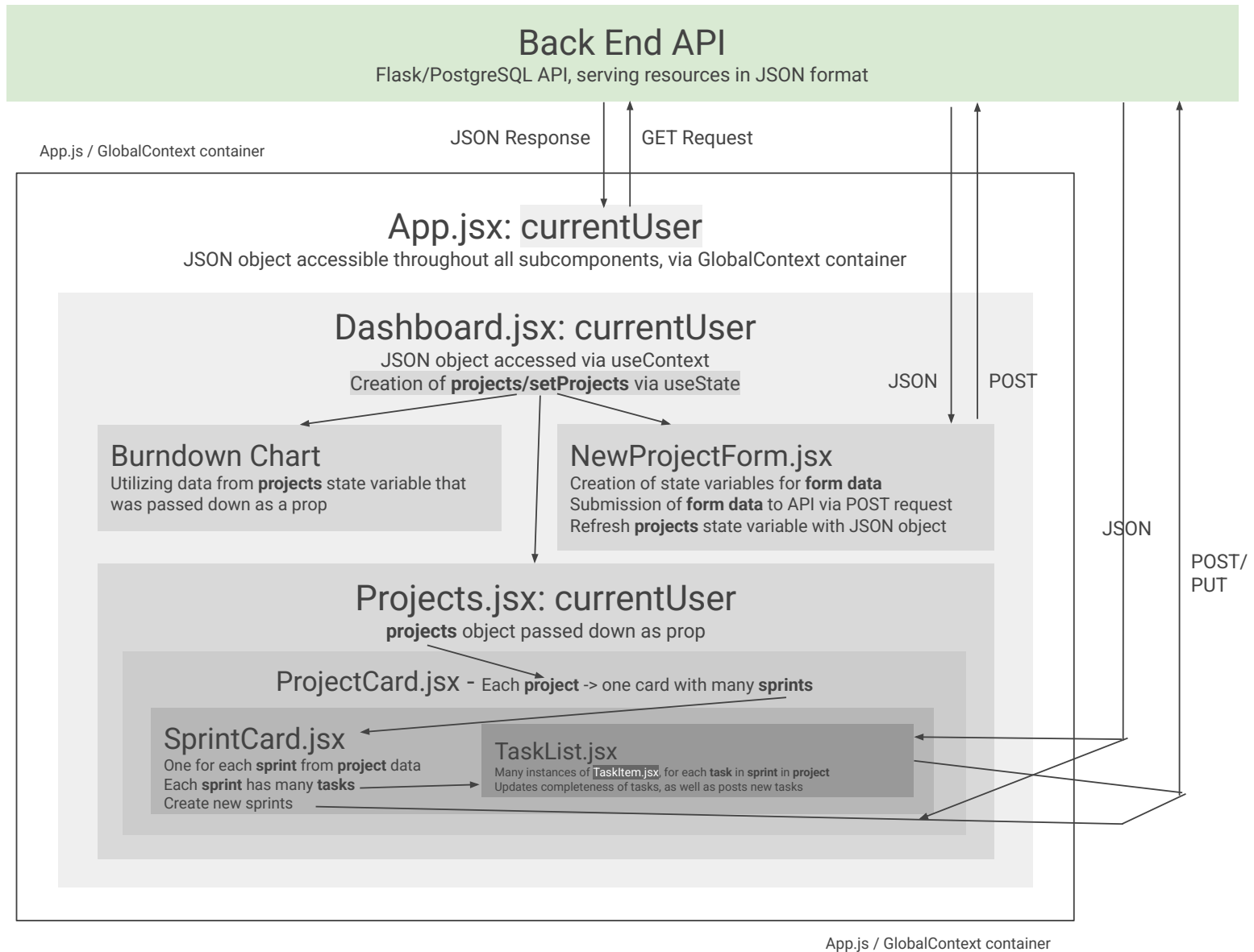


Burndown Chart
showing sprint
progress over time

Major Technical Challenges

1. Headless API Deployment
 - a. Creates hurdles in information flow/interpretation between different systems
2. State management in React (*diagram on next slide*)
 - a. Component trees and resultant API fetches can create challenges in info management
3. Render Deployment
 - a. Slow spin up time can postpone testing by a few minutes
 - b. Deployment sometimes crashes on a limited free resource such as Render
4. Local PostGres Database (pgAdmin)
 - a. Trying to create a local database is a convoluted process
5. CI/CD workflow debugging

Diagram: Front End State Variable Flow of Information



What Went Well

- Team communication using Discord, weekly meetings, and Jira for task tracking
- Effective sprint-based collaboration with regular weekly updates and reviews
- Continuous integration of work through coordinated frontend/backend development
- Informal usability testing across components improved overall system design
- Rotating Scrum Master role helped distribute leadership and coordination

What Didn't Go Well

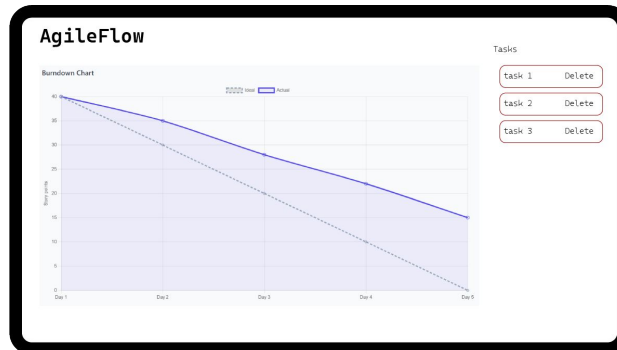
- Frontend and backend integration took longer than expected
- Delays in initial sprint planning while team roles were being finalized
- Some Learning curves with different programming languages
- Some tasks required rework during integration and testing phases
- Overall, we did not accomplish all our goals in terms of building out components
 - Burndown chart is not connected to the data
 - Account creation/login authentication with user sessions is not built out

Lessons Learned

- Using Jira helped improve sprint organization and task tracking
- GitHub branching and pull requests are critical for team-based development
- Clear role division (frontend/backend) improves development efficiency
- Clear communication prevents technical drift
- Agile sprint structure improves adaptability and project flow

Future Work

- If we continued development, we would add:
 - Messaging in between users
 - Push notifications
 - Calendar integrations
 - Rich messaging features
 - Detailed user profiles



Agile Flow

Free Agile productivity tool

- Personalization and Customization
- Enhanced Security and Privacy
- Convenience and Speed
- Loyalty and Rewards
- Improved Collaboration and Identity

AgileFlow

Sign Up

Name

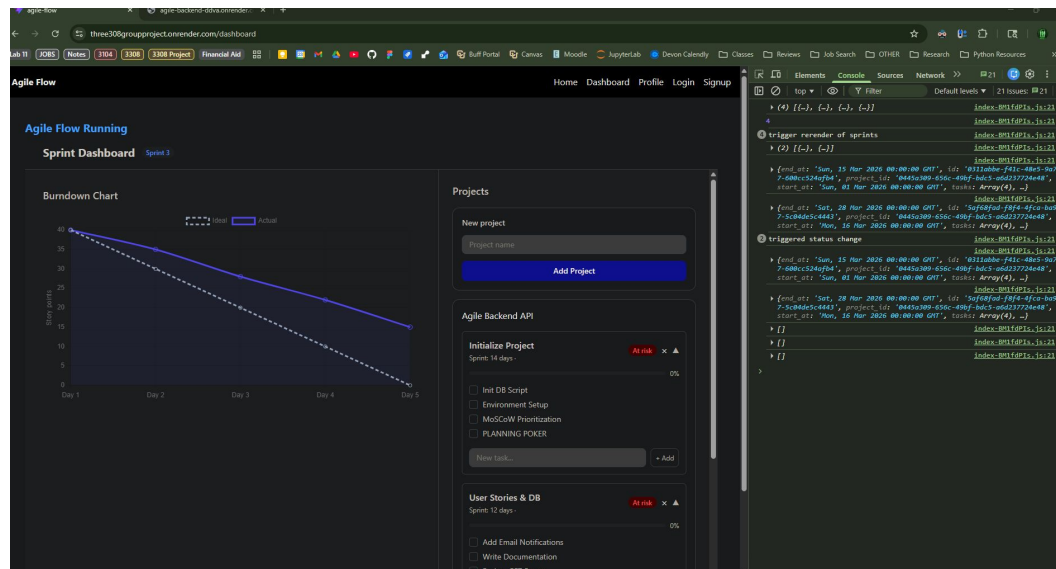
email

Password

Create Account

Final Demo Highlights

- We will highlight:
 - Demonstration of successful fetches, logging various data objects in the console
 - Smooth navigation of site, with sleek Tailwind CSS UI elements
 - Dynamic population of page information using fetched data
 - Effective CRUD operations on multiple data models, with persistence after refresh



Final Reflection

- We learned how to:
 - Plan and deliver iterative milestones
 - Collaborate using GitHub effectively
 - Evaluate architectural tradeoffs
 - Prepare a polished full-stack project
 - Present technical work professionally

Thank You

- Questions?
- GitHub:
 - Front End: <https://github.com/Agile9-3308/3308GroupProject>
 - Back End: <https://github.com/Sergrojas29/Agile-Backend>
 - Deployed Front End: <https://three308groupproject.onrender.com/>
 - Deployed Back End: <https://agile-backend-ddva.onrender.com/>